EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Support de cours - Programmation orientée objet - Java

Module LAM

Lambdas et références de méthodes

Passer un comportement aussi simplement qu'une valeur

EPHEC — Bachelier en informatique, niveau 2

Une méthode reçoit d'habitude des valeurs : un nombre, une chaîne, un objet. Ce module montre comment lui passer un comportement, c'est-à-dire ce qu'il faut faire, aussi simplement qu'on lui passe une valeur.

Vous savez déjà le faire, avec une classe qui implémente une interface. Ce module n'ajoute aucune capacité : il ajoute une écriture qui rend la chose assez légère pour qu'on la fasse vraiment, et partout. À l'examen, les lambdas reviennent surtout dans la question `JavaFX` : chaque bouton reçoit le comportement à déclencher quand on clique.

> **À la fin de ce module, vous saurez**
>
> - écrire une classe anonyme, et dire ce qu'elle a de plus et de moins qu'une classe nommée ;
> - réécrire une classe anonyme en lambda, quand l'interface est fonctionnelle ;
> - expliquer ce qu'est le type cible, et pourquoi il décide de tout ;
> - écrire une expression lambda sous ses différentes formes ;
> - choisir parmi les interfaces de `java.util.function` avant d'écrire la vôtre ;
> - remplacer une expression lambda par une référence de méthode, et donner un nom à un comportement.

LAM §1 — Une classe qui perd son nom

Vous passez déjà des comportements

```
public class TitreLong implements Predicate<String> {

    private static final int LONGUEUR_MAX = 20;

    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;    // la seule ligne utile
    }
}
```

```
titres.removeIf(new TitreLong());
```

Vous avez écrit ce code au module `COL` (voir `COL §5`). Regardez-le à nouveau, en vous demandant ce que `removeIf` reçoit vraiment. Un objet, oui. Mais elle ne lui demande rien d'autre que d'appeler `test`. Ce qu'on lui transmet, c'est un comportement : la façon de décider si un titre est trop long.

Pour transmettre cette seule ligne, il a fallu un fichier, une classe, un nom, une annotation, une signature. Quand le critère sert souvent, ce prix se justifie : la classe porte un nom, et on peut la tester seule. Quand il ne sert qu'une fois, le prix devient dissuasif. On finit par écrire une boucle à la place, et le critère se perd dans la boucle.

La classe anonyme

```
titres.removeIf(new Predicate<String>() {
    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
});
```

La première chose à retirer, c'est le nom. Ce code déclare une classe qui implémente `Predicate<String>`, et crée aussitôt son unique objet. La classe n'a pas de nom, d'où son

appellation : *classe anonyme*. Elle est écrite à l'endroit même où l'on crée son objet, et ne servira qu'ici.

Lisez-la de l'extérieur vers l'intérieur. `new Predicate<String>()` ne crée pas un objet de l'interface, ce qui est impossible depuis le module `ITF` : les accolades qui suivent sont le corps d'une classe qui l'implémente. Le mot `new` peut aussi être suivi d'une classe abstraite, dont la classe anonyme hérite alors.

La constante `LONGUEUR_MAX` vit désormais dans la classe qui contient cet appel : une classe anonyme voit les champs de la classe où elle est écrite. Le fichier a disparu, mais regardez ce qui reste autour de la ligne utile : cinq lignes « de cérémonie ».

> **Démonstration** - `Demo_LAM` : ce qu'une lambda peut remplacer, et ce qu'elle ne peut pas
>
> La démonstration part de deux classes anonymes qui se ressemblent comme deux gouttes d'eau. L'une implémente une interface, l'autre hérite d'une classe abstraite. IntelliJ propose de transformer la première, pas la seconde. Ne retenez pas le raccourci, retenez la question : qu'est-ce que l'outil sait, et que nous n'avons pas encore dit ? La réponse tient en une idée, le type cible. Retenez aussi le geste de la fin : une lambda et une classe anonyme sont deux écritures de la même chose, et IntelliJ passe de l'une à l'autre dans les deux sens. Quand une lambda vous semble obscure, dépliez-la.

LAM §2 — Ce que le compilateur sait déjà

La lambda : ce qui reste

```
titres.removeIf(new Predicate<String>() {    // impose par removeIf
    @Override
    public boolean test(String titre) {       // la seule methode a ecrire
        return titre.length() > LONGUEUR_MAX;
    }
});
```

```
titres.removeIf( titre -> titre.length() > LONGUEUR_MAX );
```

Reprenez la classe anonyme ligne par ligne, et demandez-vous ce que le compilateur savait déjà :

- que l'interface est `Predicate<String>` : c'est le type du paramètre de `removeIf` ;
- que la méthode est `test` : c'est la seule méthode abstraite de l'interface ;
- que `titre` est une `String`, et que le résultat est un `boolean` : c'est la signature de `test`.

Tout cela s'efface. Il reste ce que le compilateur ne peut pas deviner : le paramètre qu'on passe à la méthode, et ce que la méthode en fait. La flèche les sépare. C'est une *expression lambda*.

Le programme fait exactement la même chose qu'avec `TitreLong`. Seul le coût d'écriture a changé, et avec lui l'envie de le faire.

Rappel : l'interface fonctionnelle

```
@FunctionalInterface
public interface Predicate<T> {      // java.util.function

    boolean test(T t);               // la seule methode abstraite

    // ... plus des methodes default, qui ne comptent pas
}
```

Le module `ITF` a défini le terme (voir `ITF §6`). Une interface est dite *fonctionnelle* quand elle déclare une seule méthode abstraite. Elle peut avoir en plus des méthodes `default` ou `static` : elles ont déjà un corps, et ne comptent donc pas.

C'est ce qui rend la lambda possible. Une lambda n'apporte qu'un corps de méthode, sans nom. Pour que le compilateur sache quelle méthode elle écrit, il n'en faut qu'une à écrire. L'annotation `@FunctionalInterface` n'est pas obligatoire, mais elle protège : si quelqu'un ajoute une seconde méthode abstraite à l'interface, le compilateur refuse, au lieu de casser toutes les lambdas qui l'utilisent.

Le type de la lambda vient du contexte

```
// la variable fixe le type de la lambda
Predicate<String> estLong = titre -> titre.length() > LONGUEUR_MAX;

// le parametre de la methode assertThrows fixe le type de la lambda :
// Executable, une seule methode, execute()
assertThrows(IllegalArgumentException.class, () -> salle.reserver(0));
```

Une lambda seule ne veut rien dire. Le compilateur la comprend en regardant où elle va : la variable qui la reçoit, ou le paramètre auquel on la passe. Ce type de destination s'appelle le *type cible*, et c'est lui qui dit quelle méthode la lambda implémente.

Voilà la règle que la démonstration a fait trouver : le type cible doit être une interface fonctionnelle. S'il avait deux méthodes abstraites, le compilateur ne saurait pas laquelle la lambda écrit. Si c'était une classe abstraite, il faudrait un objet complet, avec son état et son constructeur, alors qu'une lambda n'apporte qu'une méthode.

Le module `JUN` vous avait demandé de recopier `() -> salle.reserver(0)` sans explication (voir `JUN §3`). Le paramètre d'`assertThrows` a pour type `Executable`, une interface de `JUnit` dont la seule méthode, `execute`, ne reçoit rien et ne retourne rien : d'où les parenthèses vides. La lambda emballe l'appel, et `assertThrows` l'exécute quand elle est prête à attraper l'exception.

Une lambda est donc un objet comme un autre : on la range dans une variable, on la passe en paramètre, on la met dans un tableau.

Les formes d'une lambda

```
Runnable saluer = () -> System.out.println("Bonjour");

Comparator<String> parLongueur = (a, b) -> Integer.compare(a.length(), b.length());

Predicate<String> estLong = titre -> {
    String sansEspaces = titre.strip();
    return sansEspaces.length() > LONGUEUR_MAX;
};
```

La forme la plus courante a un seul paramètre et une seule expression. Les autres suivent la signature de la méthode à écrire.

- **Aucun paramètre :** la méthode `run` de `Runnable` ne reçoit rien, et la lambda commence par des parenthèses vides.
- **Deux paramètres :** la méthode `compare` de `Comparator` en reçoit deux, qu'on nomme entre parenthèses, séparés par une virgule. C'est le `ParLongueur` de `COL §6`, en une ligne.
- **Plusieurs instructions :** dès que le corps compte plus d'une instruction, il prend des accolades, et le `return` redevient obligatoire, comme dans une méthode.

Un corps en bloc est permis, mais s'il s'allonge, c'est le signe qu'il mérite un nom. `LAM §3` dit comment le lui donner.

> **Exercices** - `LAM.1` Un tableau de comportements, puis `LAM.2` Écrire son interface fonctionnelle
>
> `LAM.1` range quatre lambdas dans un tableau, et les applique sans savoir ce qu'elles font. Comparez avec `ITF.1` : la même idée y demandait un fichier par calcul. Faites l'expérience de la deuxième question, en ajoutant une méthode à l'interface : regardez sur quelle ligne le compilateur proteste. `LAM.2` passe de l'autre côté : vous écrivez la méthode qui reçoit le critère, et l'interface qui le décrit, avec `@FunctionalInterface`. Le test lui passe des lambdas qu'elle n'a jamais vues, et elle s'en sort sans les connaître. Essayez vraiment la question sur la variable locale : le compilateur a un avis. Pour déplier une lambda, voir `IDE.15` ; pour nommer ce qui n'a pas de nom, `CC-MET.1`.

LAM §3 — Réutiliser ce qui existe

Les interfaces de `java.util.function`

Interface	Méthode	Elle sait
`Predicate<T>`	`boolean test(T)`	répondre oui ou non
`Function<T, R>`	`R apply(T)`	transformer un `T` en `R`
`Consumer<T>`	`void accept(T)`	faire quelque chose d'un `T`
`Supplier<T>`	`T get()`	fournir un `T`

Avant d'écrire une interface fonctionnelle, regardez si le JDK ne l'a pas déjà écrite. Le package `java.util.function` en contient une quarantaine, et ces quatre-là couvrent la plupart des besoins. Vous connaissez les deux premières depuis le module `COL`.

Toutes sont génériques, comme au module `GEN`. Pour les types primitifs, le package contient aussi des variantes qui évitent de passer par `Integer`, comme `IntPredicate` : une question de `LAM.2` vous invite à les chercher. Écrire sa propre interface reste utile quand son nom porte un sens métier ; c'est alors pour le nom, pas pour la signature.

Les méthodes de `COL`, sans classe

```
titres.removeIf(titre -> titre.length() > LONGUEUR_MAX);

titresParAuteur.computeIfAbsent("Zola", auteur -> new ArrayList<>()).add("Germinal");

titres.forEach(titre -> System.out.println(titre));
```

Au module `COL`, `removeIf` et `computeIfAbsent` recevaient des objets de classes nommées, `TitreLong` et `NouvelleListe`. Chacune tient désormais sur une ligne, à l'endroit même où elle sert, et le lecteur voit le critère sans ouvrir un autre fichier.

La méthode `forEach`, que possède toute collection, reçoit un `Consumer` et l'applique à chaque élément, dans l'ordre : c'est la boucle `for-each`, écrite comme un appel de méthode.

Remarquez le paramètre `auteur`, inutile ici puisqu'une liste vide ne dépend de personne. La signature d'`apply` l'impose, comme au module `COL` : la lambda doit le nommer, même sans s'en servir.

La référence de méthode

```
public class Titres {
    private static final int LONGUEUR_MAX = 20;

    public static boolean estLong(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
}
```

```
titres.removeIf(titre -> Titres.estLong(titre));
titres.removeIf(Titres::estLong);
```

Dernière manière de raccourcir le code. La lambda de la première ligne ne fait rien d'autre que transmettre son paramètre à une méthode qui existe : le nom `titre` entre et ressort aussitôt. On peut alors désigner la méthode elle-même : la classe, deux deux-points, et le nom, sans parenthèses.

`Titres::estLong` n'appelle pas la méthode : il la désigne, pour que `removeIf` l'appelle plus tard, sur chaque titre. Les deux lignes produisent le même objet, un `Predicate<String>`. La condition est la même que pour une lambda : la méthode doit correspondre à la méthode abstraite du type cible. Ici, `estLong` reçoit une `String` et retourne un `boolean`, comme `test`.

Le chemin est complet : une classe nommée, puis une classe sans nom, puis une lambda, puis le seul nom d'une méthode. À chaque étape, la même idée, avec moins à écrire. Entre la lambda et la référence, la différence n'est plus le coût, c'est le nom : une méthode nommée se lit, se réutilise et se teste seule.

> **Pour aller plus loin** - un stream, en passant
>
> Un *stream* fait passer les éléments d'une collection par une suite d'étapes, chacune décrite par une lambda. `titres.stream().filter(titre -> titre.length() > LONGUEUR_MAX).map(titre -> titre.toUpperCase()).collect(Collectors.joining(", "))` ouvre le flux, ne garde que les titres longs (`filter` reçoit un `Predicate`), les met en majuscules (`map` reçoit une `Function`), puis les rassemble en une seule chaîne. Avec quatre titres, « Nana », « La Débâcle », « Germinal » et « Le Docteur Pascal », et une limite de 8 caractères, il en sort « LA DÉBÂCLE, LE DOCTEUR PASCAL ». Contrairement à `removeIf`, la liste d'origine n'est pas modifiée : le stream produit un résultat neuf. Pour un tableau d'entiers, `Arrays.stream(tableau)` ouvre le même enchaînement, et `mapToObj(String::valueOf)` transforme les `int` en chaînes avant de les joindre.

> **Exercices** - `LAM.3` Celle du JDK, et les références de méthodes
>
> L'exercice répond à la question gardée depuis `LAM.2` : votre interface `Filtrage` avait déjà son équivalent dans le JDK. Remplacez-la, et constatez que rien d'autre ne change. La seconde partie fait le dernier dépouillement : trois critères écrits comme des méthodes statiques, chacune sur une ligne, puis passés par référence. Le test vérifie chaque critère seul avant le rapport : c'est ce que le nom permet, et que la lambda ne permet pas. Les questions de fin ouvrent sur les autres formes de référence de méthode. Voir `CC-NOM.1` et `CC-MET.3`.

Vous les retrouverez au module FX

```
bouton.setOnAction(evenement -> afficherTotal());
```

Un bouton reçoit le comportement à déclencher au clic. Le type cible est `EventHandler`, une interface fonctionnelle que le module `ITF` citait déjà : sa méthode `handle` reçoit l'événement, et la lambda dit quoi faire quand il survient.

Sans lambda, chaque bouton demanderait une classe, anonyme ou non. Avec elle, l'interface graphique se lit ligne par ligne : tel bouton, tel effet. C'est ce qui rend le code `JavaFX` lisible, et c'est à l'examen que vous en aurez besoin : la question `JavaFX` demande d'écrire ces gestionnaires.

> **Exercices** - `G.32` Une fabrique sans `switch` (fil rouge GameHub)
>
> L'incrément reprend la fabrique de `G.15`. Son `switch` associe chaque type de jeu à la façon d'en créer un ; `G.32` range ces associations dans une `Map`, dont chaque valeur est un `Supplier<Jeu>` : une lambda qui, à chaque appel, fabrique un jeu neuf. Ajouter un jeu revient alors à ajouter une ligne au catalogue. Répondez d'abord à la question qui compte : pourquoi ranger la façon de fabriquer un jeu, et pas le jeu lui-même ? Relisez aussi `G.17` : `StrategieNim` est fonctionnelle, et `StrategieTimide` tiendrait en une lambda, `(pile, regles) -> 1`.

Ce qu'il faut retenir

- Passer un comportement, c'est passer un objet dont on n'attend qu'une méthode.
- Une classe anonyme se déclare et s'instancie au même endroit, sans nom.
- Une lambda écrit la seule méthode d'une interface fonctionnelle : le paramètre, une flèche, le corps.
- Son type vient du contexte : le type cible, qui doit être une interface fonctionnelle.
- `Predicate`, `Function`, `Consumer`, `Supplier` : regardez dans `java.util.function` avant d'écrire votre interface.
- Une référence de méthode, `Classe::methode`, désigne une méthode qui existe ; elle donne un nom au comportement.

Le module n'ajoute aucune capacité au langage. Il en change le coût, et un comportement qui ne coûte plus qu'une ligne se passe enfin partout où il le faut.